

Computer Hardware ■ Lecture 8: Practice with Threads

Task 9: At most 3 decoders at the same time

You have just closed the ticket of Task 8. The next morning you find this e-mail:

✉ Inbox

From: Tomasz M. (Senior Software Engineer, Video Drivers)
To: you
Cc: Video Drivers Team
Subject: RE: [VD-1482] High CPU usage in the display module: semaphores as an alternative

Hi,

first of all congratulations, and thanks for closing VD-1482 so quickly. I did the review: CPU usage at idle dropped practically to zero, and QA confirms that the test laptops no longer get hot. The customer has been informed too. Good job.

While we are at it, I'd like to point you to one more tool that you will meet quite often in our code base: **semaphores**. Here is a short summary, so that we have the same context.

What a semaphore is. It is a counter that can never go below zero. It has two operations:

- `acquire()`: if the counter is greater than zero, decrease it by 1 and return at once. If the counter is 0, the thread sleeps until somebody increases it.
- `release()`: increase the counter by 1 and wake up one of the waiting threads.

Fun fact: semaphores were invented by Dijkstra, the same person who described producer-consumer, and in the same work from 1965 he solved that problem with them.

How to use it in C++. Since C++20 there is `std::counting_semaphore<N>` in the header `<semaphore>`. An example from our own backyard: the GPU has 3 hardware decoding units, so at most 3 streams can be decoded at the same time.

```
std::counting_semaphore<3> hw_decoders{3};    // 3 free units at the start

void decode_stream(int id) {
    hw_decoders.acquire();    // take a unit; if all are taken, sleep
    decode(id);
    hw_decoders.release();    // give the unit back
}
```

How it differs from a mutex. A mutex is in practice a semaphore with a counter of 1, with one important difference: only the thread that locked a mutex may unlock it. `release()` of a semaphore may be called by **any** thread. That is why a semaphore is good not only for limits but also for signalling: one thread calls `release()`, another one waits in `acquire()`.

When a semaphore, and when a condition variable. If you wait for a simple count ("there are free places", "there is something to do"), a semaphore usually gives shorter code. If the condition is more complex, stay with a condition variable.

So I have two requests for this sprint:

1. **[VD-1497]** We have reports that with several videos at once the driver returns a hardware error, because it tries to use a fourth decoding unit, which does not exist. Please make sure that at most 3 streams are decoded at the same time.
2. **[VD-1503]** Try to rewrite the buffer from VD-1482 with two semaphores: one counts the free places, the other one the ready frames. I'd like to compare both versions at the next code review.

If anything is unclear, just write, or catch me at the daily.

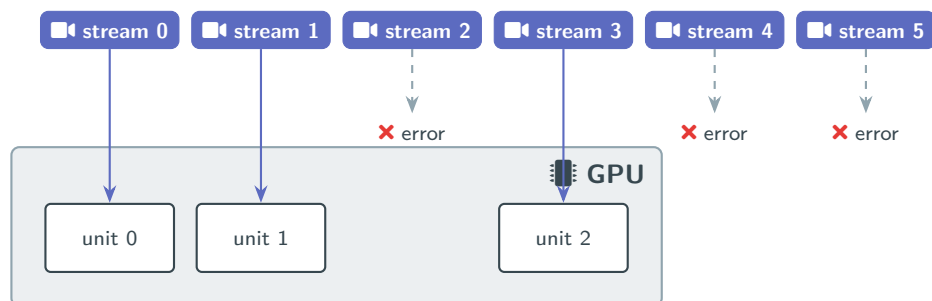
Best regards,
Tomasz

Tomasz M.
Senior Software Engineer ■ Video Drivers Team

This task is ticket VD-1497. Task 10 is ticket VD-1503.

The task: VD-1497

Six videos play at the same time, each one in its own thread. The GPU has only 3 decoding units. The ready function `decode()` checks how many units are in use and reports a hardware error when somebody tries to use a fourth one.



6 streams want to decode at the same time, the GPU has only 3 units

Ready code (in `gpu.h`, we do not look inside)

Functions

<code>decode(id)</code>	decode one stream on a free unit: 1 s; print an error if all 3 units are in use
<code>report()</code>	print how many hardware errors there were

Constants

<code>UNITS</code>	3 decoding units on the GPU
<code>STREAMS</code>	6 videos at the same time

The template is in [Task9](#). This is `streams.cpp`:

```
#include <thread>
#include <vector>
#include "gpu.h"

void decode_stream(int id) {
    decode(id);
}

int main() {
    std::vector<std::thread> threads;
    for (int id = 0; id < STREAMS; id++)
        threads.emplace_back(decode_stream, id);
    for (auto &t : threads)
        t.join();
    report();
}
```

In the task folder, make run compiles the program and runs it.

Your tasks

- 1 Run it with `time ./build/streams`. How many streams are decoded at the same time? How many hardware errors are there?
- 2 Add a `std::counting_semaphore` so that at most 3 streams are decoded at the same time. How long does the whole program take now?
- 3 A colleague puts one mutex around `decode()` instead of the semaphore. Do the errors go away? How long does the program take then? Why?
- 4 Somebody forgets `release()` in one place (for example on a path that ends the function early). What happens after a few videos?